- *Creates a JWT.* By default, the token is created with a `sub` and `unique_claim` set to your operating system's username, with `aud` claims for each of the `applicationUrls` in your launchSettings.json and an issuer of `dotnet-user-jwts`. You'll notice that these match the values added to your APIs configuration file.

  After calling `dotnet user-jwts create`, the JWT token is printed to the console, along with the `sub` name used and the ID of the token. I've truncated the tokens throughout this chapter for brevity:

```
New JWT saved with ID 'f2080e51'.
Name: andrewlock

Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmlxdWVfbmFtZSI6ImFuZHJl...
```

TIP You can visualize exactly what's in the token by copy and pasting it into https://jwt.io, as I showed in figure 25.7.

Now that you have a token, it's time to test it. To use the token, you need to add an `Authorization` header to requests using the following format (where `<token>` is the full token printed by user-jwts):

```
Authorization: Bearer <token>
```

If any part of this header is incorrect—if you misspell `Authorization`, misspell `Bearer`, don't include a space between `Bearer` and your token, or mistype your token—you'll get a `401 Unauthorized` response.